



VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi – 590 018

MINI PROJECT REPORT

On

PROMPT OPTIMIZER

Submitted in partial fulfillment of the requirement for the curriculum of the 5th Semester

Bachelor of Engineering

in

COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

Submitted by

ANUSHA M : 1VI23CD004

ARCHANA R : 1VI23CD005

KHUSHI M D : 1VI23CD023

VYSHNAVI R K : 1VI23CD061

Under the guidance of

Dr.A Rosline Mary

Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

VEMANA INSTITUTE OF TECHNOLOGY

BENGALURU – 560034

2025 - 2026

CERTIFICATE

This is to certify that the Mini Project work entitled “**PROMPT OPTIMIZER**” carried out by **ANUSHA M (1VI23CD004), ARCHANA R (1VI23CD005), KHUSHI M D (1VI23CD023), and VYSHNAVI R K (1VI23CD061)** is a bonafide work submitted in partial fulfillment for the award of Bachelor of Engineering in Computer Science and Engineering (Data Science) of Visvesvaraya Technological University, Belagavi during the academic year 2025–2026.

It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of the mini project work prescribed for the said degree.

Guide

HOD

Principal

Prof.A Rosline Mary

Dr. Mamatha C R

Dr. Vijayasimha Reddy B G

ACKNOWLEDGEMENT

We sincerely thank Visvesvaraya Technological University for providing an opportunity to carry out this mini project.

We express our deep sense of gratitude to our institution, Vemana Institute of Technology, Bengaluru, for providing the facilities and support required for successful completion of the project.

We would like to thank Dr. Vijayasimha Reddy B G, Principal, Vemana Institute of Technology, Bengaluru for providing encouragement and institutional support.

We would also like to place on record our sincere gratitude to Dr. Mamatha C R, Head of the Department, Computer Science and Engineering (Data Science), for her valuable guidance and continuous support.

We express our heartfelt thanks to our project guide Prof. Krishna V V, Assistant Professor, Department of Computer Science and Engineering (Data Science), for her constant encouragement, guidance, and valuable suggestions throughout the project.

We would also like to thank all faculty members, friends, and family members for their support and encouragement during the completion of this project.

Date: 15-05-2026

Place: Bengaluru

ABSTRACT

Prompt Optimizer is an intelligent Generative AI-based system designed to improve the quality of prompts provided by users before sending them to an AI model. The system analyzes user input, identifies ambiguity, grammatical issues, and missing context, and then transforms the prompt into a more detailed and structured version.

The project integrates Natural Language Processing (NLP) techniques and prompt engineering methods to generate optimized prompts that improve the accuracy and relevance of AI-generated responses. By enhancing clarity, specificity, and context, the system helps users communicate more effectively with Large Language Models (LLMs).

The Prompt Optimizer can be used in various domains such as education, content creation, chatbot systems, coding assistance, and research applications. The system demonstrates how Generative AI and NLP can improve human-computer interaction and increase the efficiency of AI systems.

Keywords: Generative AI, Prompt Engineering, NLP, Artificial Intelligence, Prompt Optimization.

TABLE OF CONTENTS

Chapter No.	Chapter Title	Page No.
ACKNOWLEDGEMENT		i
ABSTRACT		ii
LIST OF FIGURES		iii
LIST OF TABLES		iv
1	INTRODUCTION	1
1.1	Background and Problem Domain	1
1.2	Proposed Solution	2
1.3	Project Objectives	2
2	LITERATURE REVIEW	3
2.1	Introduction to Literature Review	3
2.2	Review of Existing Systems	3
2.3	Advantages of Existing Systems	4
3	SYSTEM ANALYSIS	5
3.1	Requirement Analysis	5
3.1.1	Functional Requirements	5
3.1.2	Non-Functional Requirements	6
4	SYSTEM TESTING	7
4.1	Screenshots of Key Modules	7
5	CONCLUSION AND FUTURE SCOPE	12
5.1	Conclusion	12
5.2	Future Scope	13

LIST OF FIGURES

Figure No.	Figure Title
Figure 4.1	Home Page
Figure 4.2	Prompt Input Module
Figure 4.3	Prompt Classification and Detection
Figure 4.4	Optimized Prompt Generation
Figure 4.5	AI Response Generation
Figure 4.6	Full Answer Output
Figure 4.7	Explanation and Edge Case Handling

LIST OF TABLES

Table No.	Table Title
Table 3.1	Functional Requirements
Table 3.2	Non-Functional Requirements

1. INTRODUCTION

Prompt Optimizer is a Generative AI-based application developed to improve the quality of prompts entered by users before they are processed by Artificial Intelligence systems. In recent years, Large Language Models (LLMs) and Generative AI applications have become widely popular for tasks such as content generation, coding, learning assistance, and chatbot communication.

However, the quality of AI-generated responses highly depends on the quality of the input prompt. Many users provide unclear, short, or ambiguous prompts, which leads to inaccurate or incomplete responses. To solve this issue, the Prompt Optimizer system automatically enhances user prompts using NLP techniques and prompt engineering strategies.

The system analyzes the prompt, identifies missing context, improves sentence structure, corrects grammar, and generates an optimized prompt that produces better AI outputs.

1.1 Background and Problem Domain

Generative AI systems require detailed and properly structured prompts to generate meaningful responses. Most users are unaware of prompt engineering techniques and often provide insufficient instructions to AI models.

Some common problems include:

- Lack of clarity in prompts
- Incomplete instructions
- Ambiguous keywords
- Grammatical errors
- Missing contextual information

These problems reduce the efficiency and accuracy of AI systems. Therefore, there is a need for an intelligent system that can automatically optimize prompts before they are processed by AI models.

1.2 Proposed Solution

The proposed system introduces a Prompt Optimizer that improves user prompts using Natural Language Processing and Generative AI techniques.

The system performs the following tasks:

- Analyzes user prompts
- Identifies missing context
- Corrects grammatical issues
- Expands keywords
- Enhances prompt clarity
- Generates optimized prompts

The optimized prompts are then sent to the AI model to generate accurate and meaningful responses.

1.3 Project Objectives

The main objectives of the project are:

- To improve the quality of prompts entered by users
- To generate better AI responses
- To reduce ambiguity in prompts
- To simplify communication with AI systems
- To enhance the performance of Generative AI applications
- To develop a user-friendly AI assistance system

2. LITERATURE REVIEW

2.1 Introduction to Literature Review

The literature review provides an understanding of existing research and technologies related to Generative AI, Prompt Engineering, and Natural Language Processing.

Prompt engineering has become an important area in AI systems because the performance of Large Language Models depends greatly on the structure and clarity of prompts.

Researchers have proposed various techniques to improve prompts manually and automatically. NLP-based systems have also been used to analyze sentence structures, grammar, and contextual meaning.

2.2 Review of Existing Systems

Existing AI systems and chatbot applications provide direct responses based on user prompts without improving the prompt quality.

Some existing systems include:

- AI chatbots
- NLP grammar correction tools
- Text enhancement applications
- AI writing assistants
- Prompt engineering frameworks

Limitations of existing systems:

- Lack of automatic optimization
- Limited contextual understanding
- No intelligent prompt restructuring
- Dependency on user prompt quality

2.3 Advantages of Existing Systems

Advantages observed in existing systems include:

- Fast response generation
- AI-based text generation
- Basic grammar correction
- User-friendly interfaces
- Support for content generation

The proposed Prompt Optimizer improves upon these systems by providing automatic prompt enhancement and contextual optimization.

3. SYSTEM ANALYSIS

3.1 Requirement Analysis

Requirement analysis is the process of identifying the needs and functionalities required for successful implementation of the Prompt Optimizer system.

The system must efficiently process user prompts and generate optimized versions with improved clarity and detail.

3.1.1 Functional Requirements

Table 3.1 Functional Requirements

Requirement	Description
User Input	Accept prompts from users
NLP Processing	Analyze grammar and keywords
Prompt Optimization	Improve prompt clarity and context
AI Integration	Connect with AI model APIs
Response Display	Display optimized prompts and AI responses
Error Handling	Handle invalid or incomplete prompts

3.1.2 Non-Functional Requirements

Table 3.2 Non-Functional Requirements

Requirement	Description
Performance	Fast prompt processing
Reliability	Consistent optimization results
Scalability	Ability to support multiple users
Security	Safe handling of user data
Usability	Easy-to-use interface
Maintainability	Easy software updates and improvements

4. SYSTEM TESTING

4.1 Screenshots of Key Modules

This section presents screenshots of the implemented Prompt Optimizer system, highlighting the important functionalities and interfaces developed in the project.

Figure 4.1: Home Page

The home page of the Prompt Optimizer system provides a clean and user-friendly interface where users can enter prompts for optimization. The interface also displays example prompts to help users understand the functionality of the system.

Features shown:

- Prompt input area
- Example prompt suggestions
- Optimize Prompt button
- User-friendly layout



Figure 4.1: Home Page

Figure 4.2: Prompt Input Module

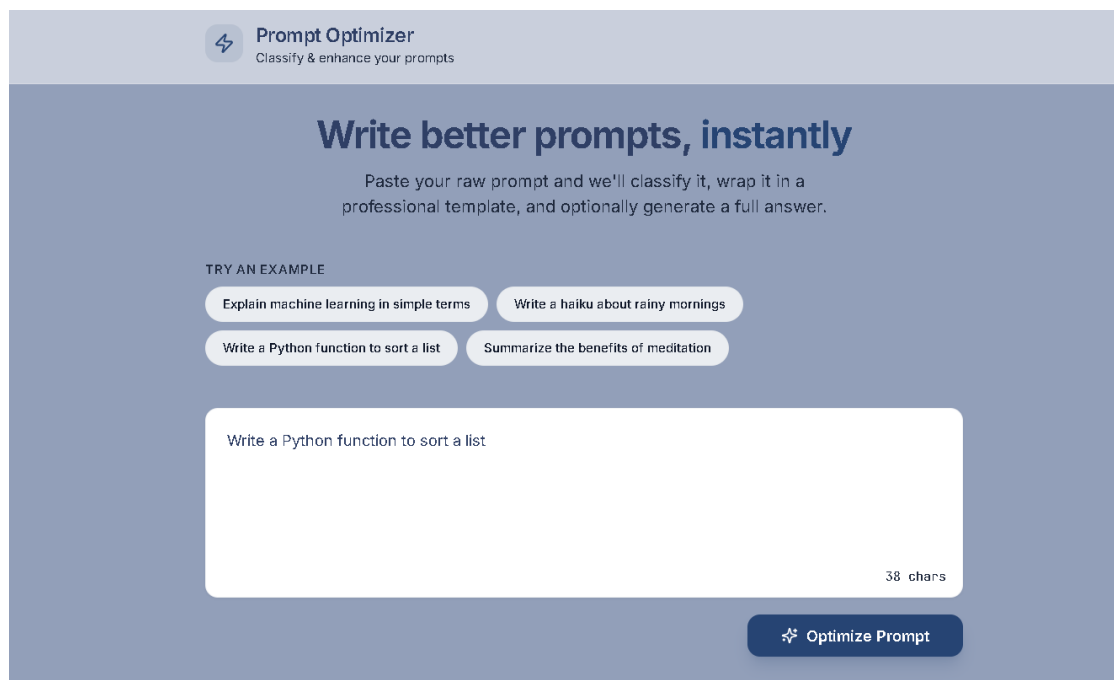
This module allows users to type or paste prompts into the system for optimization.

Example Prompt:

“Write a Python function to sort a list”

The system accepts different categories of prompts such as:

- Coding prompts
- Educational prompts
- Content generation prompts
- Creative writing prompts



The screenshot displays the 'Prompt Optimizer' web interface. At the top, a header bar contains a lightning bolt icon and the text 'Prompt Optimizer' with the subtitle 'Classify & enhance your prompts'. Below this, a main heading reads 'Write better prompts, instantly', followed by a subtext: 'Paste your raw prompt and we'll classify it, wrap it in a professional template, and optionally generate a full answer.' A section titled 'TRY AN EXAMPLE' features four buttons: 'Explain machine learning in simple terms', 'Write a haiku about rainy mornings', 'Write a Python function to sort a list', and 'Summarize the benefits of meditation'. The 'Write a Python function to sort a list' button is selected, and its text is pasted into a large white text input field. In the bottom right corner of the input field, it says '38 chars'. At the bottom right of the interface is a dark blue button with a lightning bolt icon and the text 'Optimize Prompt'.

Figure 4.2: Prompt Input Module

Figure 4.3: Prompt Classification and Detection

The Prompt Optimizer identifies the type of prompt entered by the user. In this example, the system detects the prompt category as:

Coding

This helps the system apply suitable prompt engineering techniques based on the prompt type.

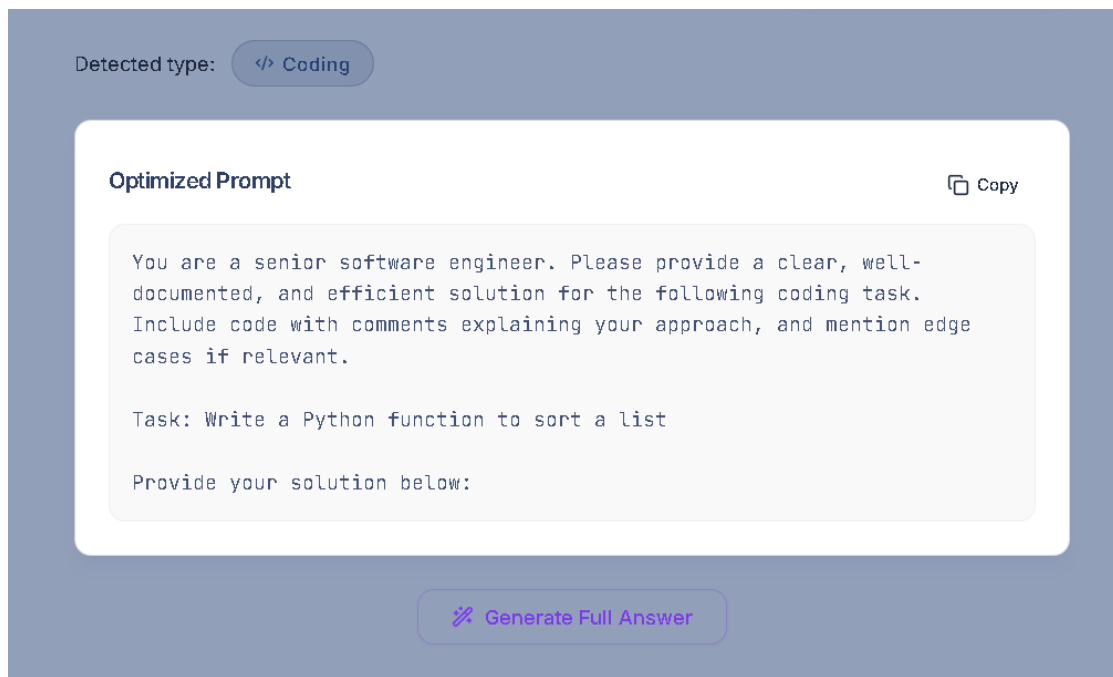


Figure 4.3: Prompt Classification and Detection

Figure 4.4: Optimized Prompt Generation

The system automatically converts the original prompt into a more professional and detailed version.

Original Prompt:

“Write a Python function to sort a list”

Optimized Prompt:

“You are a senior software engineer. Please provide a clear, well-documented, and efficient solution for the following coding task. Include code with comments explaining your approach, and mention edge cases if relevant.”

The optimized prompt improves clarity, structure, and context.

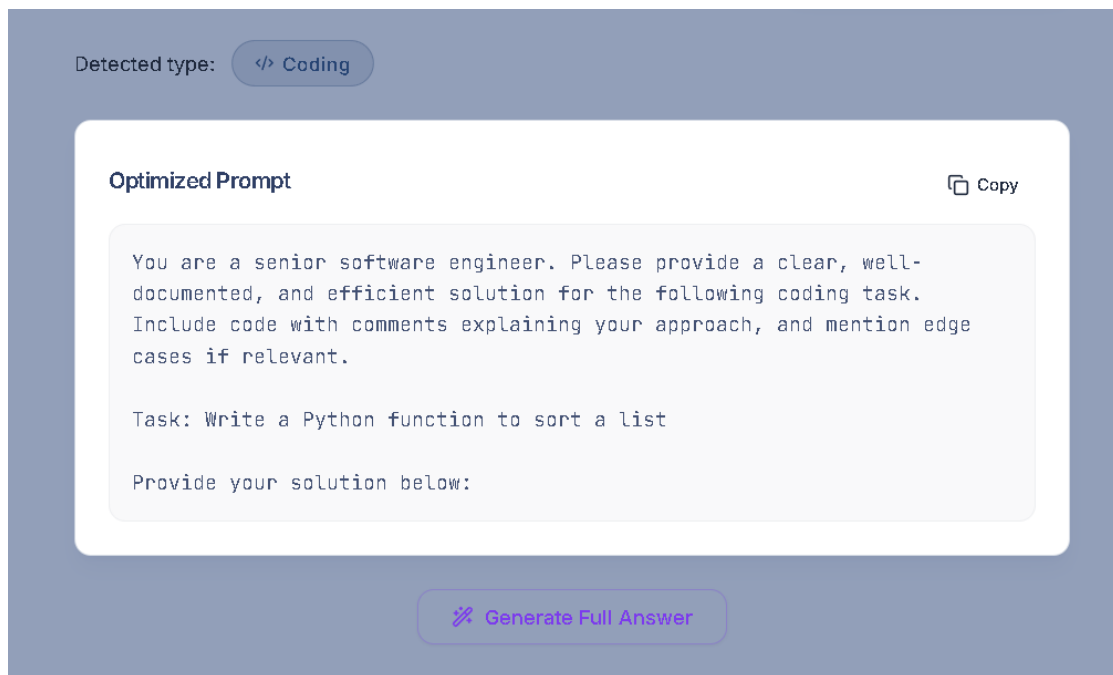


Figure 4.4: Optimized Prompt Generation

Figure 4.5: AI Response Generation

After optimization, the enhanced prompt is sent to the Generative AI model. The AI generates:

- Well-structured code
- Proper documentation
- Error handling
- Edge case explanations

The generated output includes:

- Python function implementation
- Comments
- Example outputs
- Explanation of approach

Full Answer

Certainly! Below is a well-documented Python function that sorts a list. We will use the built-in `sorted()` function in Python, which is based on Timsort and is efficient for most cases. Additionally, I'll include an explanation of the approach and edge cases.

```
def sort_list(input_list):  
    """  
        Sorts a given list of elements in ascending order.  
  
        Parameters:  
        input_list (list): The list of elements to be sorted.  
                           The elements can be of any comparable type.  
  
        Returns:  
        list: A new list containing the sorted elements.  
  
        Raises:  
        TypeError: If the elements of the input list cannot be compared.  
  
        Example:  
        >>> sort_list([3, 1, 4, 1, 5, 9])  
        [1, 1, 3, 4, 5, 9]  
  
        >>> sort_list(['banana', 'apple', 'cherry'])
```

Figure 4.5: AI Response Generation

Figure 4.6: Full Answer Output

The generated response contains:

- Python sorting function
- Docstrings
- Return values
- Error handling
- Example usage
- Edge case handling

The system ensures the AI-generated response is:

- Detailed
- Accurate
- Readable

- Beginner-friendly

```
[1, 1, 3, 4, 5, 9]

>>> sort_list(['banana', 'apple', 'cherry'])
['apple', 'banana', 'cherry']
"""

# Attempt to sort the provided list
try:
    # Use the built-in sorted() to handle sorting
    sorted_list = sorted(input_list)
except TypeError:
    # Handle the case where elements are not comparable
    raise TypeError("All elements in the list must be comparable.")

return sorted_list

: Example usage
if __name__ == "__main__":
    # Testing the function with various lists
    print(sort_list([3, 1, 4, 1, 5, 9])) # Output: [1, 1, 3, 4, 5, 9]
    print(sort_list(['banana', 'apple', 'cherry'])) # Output: ['apple',
    print(sort_list([3, 5, 2, 1, 4, 8])) # Output: [1, 2, 3, 4, 5, 8]
```

Figure 4.6: Full Answer Output

Figure 4.7: Explanation and Edge Case Handling

The system also generates explanations for:

- Function definition
- Sorting logic
- Exception handling
- Empty list handling
- Identical element handling
- Comparability of list elements

This improves the educational value of the generated response.

```
# Edge case: List of identical elements
print(sort_list([1, 1, 1])) # Output: [1, 1, 1]
```

Explanation of the Approach:

Function Definition: We define the function `sort_list` that takes a single parameter `input_list`.

Docstring: A clear docstring is provided to explain the function parameters, return type, exceptions raised, and examples of usage.

Sorting: We utilize Python's built-in `sorted()` function, which is efficient and handles various data types that are mutually comparable.

Error Handling: We include error handling for `TypeError` to catch and handle exceptions where list elements are not comparable (e.g., mixing numbers with strings).

Return Value: The function returns a new sorted list without modifying the original input list, adhering to functional programming principles.

Edge Cases Considered:

Empty List: The function should handle an empty list gracefully, returning an empty list.

Identical Elements: The function should correctly sort lists where all elements are the same.

Comparability: The type handling ensures that the function raises an appropriate error if there are elements that cannot be compared.

This function is efficient and robust for general sorting needs in Python.

Figure 4.7: Explanation and Edge Case Handling

5. CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

The Prompt Optimizer project successfully demonstrates the practical implementation of Generative AI and Natural Language Processing techniques to improve the quality of prompts provided by users.

The system analyzes prompts, detects prompt categories, improves clarity and structure, and generates optimized prompts that produce accurate and meaningful AI-generated responses.

The project highlights the importance of prompt engineering in improving the performance of Large Language Models (LLMs). By enhancing prompt quality, the system improves communication between users and AI systems.

The developed Prompt Optimizer provides several benefits such as:

- Better AI-generated responses
- Reduced ambiguity in prompts
- Improved prompt clarity
- Faster interaction with AI systems
- Enhanced user experience
- Efficient content generation

The project also demonstrates how AI can assist users not only in generating outputs but also in improving the quality of inputs provided to AI systems.

Overall, the Prompt Optimizer serves as an intelligent and user-friendly Generative AI application useful in education, coding assistance, content generation, and research support.

5.2 Future Scope

The future enhancements of the Prompt Optimizer project include:

1. Voice-based prompt optimization using speech recognition
2. Multi-language prompt enhancement support
3. Personalized prompt suggestions using user history
4. AI-based prompt scoring system

5. Real-time prompt improvement recommendations
6. Integration with multiple AI platforms
7. Mobile application support
8. Advanced analytics dashboard
9. Cloud deployment for large-scale usage
10. AI-powered grammar and contextual correction

These future improvements can make the system more intelligent, scalable, and effective for a wider range of users and AI applications.